

VAIDSYS TECHNOLOGIES

Internship Project Report

DESIGN AND VERIFICATION OF 32-BIT ARITHMETIC LOGIC UNIT USING VERILOG HDL

Submitted by:

Ayan Arif

B.Tech - Electronics Engineering (VLSI Design & Technology)

Jamia Millia Islamia, New Delhi

Batch: 2024–2028

Internship Domain: VLSI Design and Verification

Organization: Vaidsys Technologies

Tools: Xilinx Vivado ML 2025.2 | Verilog HDL | XSim Simulator

Target Device: Xilinx Artix-7 (xc7a200tfbg676-2)

Date: 15th July 2026

TABLE OF CONTENTS

1. Abstract	3
2. Introduction	3
3. Problem Statement & Objectives	5
4. ALU Architecture & Design	5
5. Opcode Table	7
6. Tools Used	7
7. Verilog Design Code (alu.v)	8
8. Testbench Code (alu_tb.v)	10
9. RTL Schematic	13
10. Simulation Results	16
11. Synthesis, Utilization and Project Summary	19
12. Timing Analysis	22
13. Power Analysis	23
14. FPGA Floor Plan	25
15. Conclusion and Future Scope	26
16. References	27

1. ABSTRACT

This project presents the design and functional verification of a 32-bit Arithmetic Logic Unit (ALU) using Verilog Hardware Description Language (HDL). The ALU supports eight operations: Addition, Subtraction, Bitwise AND, Bitwise OR, Bitwise XOR, Bitwise NOT, Logical Shift Left, and Logical Shift Right, all controlled through a 3-bit opcode. Three status flags Zero, Carry, and Overflow are implemented to indicate special arithmetic conditions.

The design targets the Xilinx Artix-7 FPGA (xc7a200tfbg676-2) and was developed using parameterized behavioral RTL modeling in Xilinx Vivado ML 2025.2. A testbench with 10 structured test cases covering all operations and edge cases was used for functional verification. All test cases produced correct results.

Post-synthesis resource utilization is 298 Slice LUTs out of 134,600 available, which is 0.22% an extremely efficient result. Timing analysis shows zero violations with infinite worst negative slack. Power analysis estimates a total on-chip power of 23.553W at typical process conditions with a junction temperature of 69 degrees Celsius, which is safely within the 85 degree Celsius limit for the Artix-7 family.

2. INTRODUCTION

The Arithmetic Logic Unit is the fundamental computational block of every digital processor. It performs all arithmetic and logical operations required by the CPU instruction set. In a pipelined processor, the ALU sits at the heart of the Execute stage and computes results for every arithmetic, logical, and shift instruction that the processor handles.

Hardware Description Languages such as Verilog HDL allow engineers to describe complex digital circuits at a high level of abstraction and verify their behavior through simulation before committing to expensive physical fabrication. This project applies these industry-standard RTL design techniques to implement a complete 32-bit ALU from scratch, following the same design flow used in professional VLSI environments.

This project was completed as part of the VLSI Design internship domain at Vaidsys Technologies. It covers the complete RTL flow from writing synthesizable Verilog, writing a

testbench, running functional simulation, performing RTL synthesis, and analyzing post-synthesis area, timing, and power reports.

3. PROBLEM STATEMENT AND OBJECTIVES

3.1 Problem Statement

Modern processors require an ALU that performs multiple arithmetic and logical operations efficiently with correct flag generation and minimal resource usage. A poorly designed ALU creates a performance bottleneck in the processor pipeline. This project focuses on the RTL design and functional verification of a 32-bit ALU that is correct for all supported operations, synthesizes efficiently on FPGA fabric, and supports proper signed arithmetic with Zero, Carry, and Overflow flag generation.

3.2 Objectives

1. Design a parameterized 32-bit ALU in behavioral Verilog HDL
2. Support 8 arithmetic and logical operations via a 3-bit opcode
3. Implement Zero, Carry, and Overflow status flags with correct Boolean logic
4. Write a structured Verilog testbench with 10 test cases covering all operations and edge cases
5. Perform functional simulation and verify all operations in Vivado XSim
6. Run RTL synthesis targeting Xilinx Artix-7 FPGA
7. Analyze and document area utilization, timing report, power report, and floor plan

4. ALU ARCHITECTURE AND DESIGN

4.1 Design Overview

The ALU is described using behavioral RTL modeling. It takes two 32-bit operands A and B and a 3-bit opcode, and produces a 32-bit result along with three 1-bit status flags. A parameterized WIDTH of 32 allows the design to be scaled to 8, 16, or 64 bits by changing a single parameter. Operation selection is handled by a case statement inside a combinational `always@(*)` block, while all flags are driven by continuous assign statements outside the `always` block for cleaner synthesis with no risk of latch inference.

4.2 Port Description

Port	Width	Direction	Description
A	32-bit	Input	First operand
B	32-bit	Input	Second operand
opcode	3-bit	Input	Operation selector 8 operations supported
result	32-bit	Output	Result of selected operation
zero	1-bit	Output	Set high when result is all zeros
carry	1-bit	Output	Unsigned overflow or borrow flag for ADD and SUB
overflow	1-bit	Output	Signed arithmetic overflow flag for ADD and SUB

4.3 Key Design Decisions

- Parameterized WIDTH=32 the design scales to any bit width with a single parameter change
- Localparam constants for all 8 opcodes eliminates magic numbers and makes code self-documenting
- 33-bit extended wires add_ext and sub_ext cleanly captures carry and borrow bits from addition and subtraction
- Zero flag uses OR-reduction ($\sim(|\text{result})$) synthesizes to a single efficient gate chain, the most compact implementation possible
- Flags computed via assign statements outside always block guarantees purely combinational flag logic with no latch inference
- Overflow detection uses Boolean expressions directly no if-else chains, cleaner gate-level mapping after synthesis

5. OPCODE TABLE

The 3-bit opcode input selects one of 8 operations. Using 3 bits provides exactly 8 unique codes (000 through 111), one per operation, with no unused codes.

Opcode	Mnemonic	Verilog Expression	Description
3'b000	ADD	result = add_ext[WIDTH-1:0]	32-bit addition, carry captured in add_ext[32]
3'b001	SUB	result = sub_ext[WIDTH-1:0]	32-bit subtraction, borrow captured in sub_ext[32]
3'b010	AND	result = A & B	Bitwise AND of all 32 bits
3'b011	OR	result = A B	Bitwise OR of all 32 bits
3'b100	XOR	result = A ^ B	Bitwise XOR of all 32 bits
3'b101	NOT	result = ~A	Bitwise complement of operand A only
3'b110	SHL	result = A << B[4:0]	Logical left shift B[4:0] used as shift amount
3'b111	SHR	result = A >> B[4:0]	Logical right shift B[4:0] used as shift amount

6. TOOLS USED

Xilinx Vivado ML 2025.2: Industry-standard FPGA design suite used for RTL entry, behavioral simulation via XSim, RTL synthesis, implementation, utilization reporting, timing analysis, and power estimation. Vivado is the standard tool in academic and professional Xilinx FPGA design workflows.

Verilog HDL (IEEE 1364): Hardware Description Language used to write the ALU design module and testbench at behavioral RTL level. Verilog describes hardware structurally and behaviorally and is the most widely used HDL in commercial VLSI design globally.

XSim Simulator: Vivado's built-in HDL simulator used to run functional behavioral simulation and generate signal waveforms for visual and console-based verification of all ALU operations.

Target Device Xilinx Artix-7 (xc7a200tfbg676-2): A high-capacity Artix-7 FPGA with 134,600 LUTs, 400 I/O pins, 740 DSP blocks, and 325 Block RAMs. This device has ample resources for the 32-bit ALU and is representative of FPGAs used in mid-range signal processing and embedded applications.

7. VERILOG DESIGN CODE (alu.v)

The ALU is described in a single parameterized Verilog module using behavioral RTL modeling. Localparam constants define all 8 opcode values. Two 33-bit extended wires capture carry and borrow. A combinational always@(*) block with a case statement handles operation selection. Three continuous assign statements handle flag generation outside the always block.

```
`timescale 1ns / 1ps

module alu #(
    parameter WIDTH = 32
) (
    input wire [WIDTH-1:0] A,
    input wire [WIDTH-1:0] B,
    input wire [2:0] opcode,
    output reg [WIDTH-1:0] result,
    output wire zero,
    output wire carry,
    output wire overflow
);

    localparam ADD = 3'b000;
    localparam SUB = 3'b001;
    localparam AND = 3'b010;
    localparam OR = 3'b011;
    localparam XOR = 3'b100;
    localparam NOT = 3'b101;
    localparam SHL = 3'b110;
    localparam SHR = 3'b111;

    wire [WIDTH:0] add_ext;
    wire [WIDTH:0] sub_ext;

    assign add_ext = {1'b0, A} + {1'b0, B};
    assign sub_ext = {1'b0, A} - {1'b0, B};

    always @ (*) begin
        result = {WIDTH{1'b0}};
    end
```

Figure 1: alu.v Module header, port declarations, localparam opcode definitions, and extended wire assignments

The module header uses parameter WIDTH=32 making the design scalable. Localparams ADD through SHR define readable opcode names. The two 33-bit wires add_ext and sub_ext pre-compute both the result and the overflow bit for addition and subtraction simultaneously using concatenation, which is efficient and avoids recomputing inside the case statement.


```

    case (opcode)
        ADD : result = add_ext[WIDTH-1:0];
        SUB : result = sub_ext[WIDTH-1:0];
        AND : result = A & B;
        OR  : result = A | B;
        XOR : result = A ^ B;
        NOT : result = ~A;
        SHL : result = A << B[4:0];
        SHR : result = A >> B[4:0];
    endcase
end

assign zero      = ~(|result);

assign carry     = (opcode == ADD) ? add_ext[WIDTH] :
                   (opcode == SUB) ? sub_ext[WIDTH] :
                   1'b0;

assign overflow  = (opcode == ADD) ?
                   ((~A[WIDTH-1] & ~B[WIDTH-1] & result[WIDTH-1]) |
                    ( A[WIDTH-1] & B[WIDTH-1] & ~result[WIDTH-1])) :
                   (opcode == SUB) ?
                   ((~A[WIDTH-1] & B[WIDTH-1] & result[WIDTH-1]) |
                    ( A[WIDTH-1] & ~B[WIDTH-1] & ~result[WIDTH-1])) :
                   1'b0;

endmodule

```

Figure 2: *alu.v* Case statement for all 8 operations, zero flag via OR-reduction, carry flag and overflow flag via assign

The case statement selects one of 8 operations based on opcode. The default assignment of result to zero at the start of the always block prevents latch inference. Flags are assigned outside using continuous assign statements the zero flag uses $\sim(|\text{result})$ which is an OR-reduction of all 32 bits followed by inversion, synthesizing to the minimum possible gate count. The carry and overflow assignments use ternary operators to check the opcode and select the correct extended wire bit or overflow condition.

8. TESTBENCH CODE (alu_tb.v)

A Verilog testbench applies stimulus to the ALU and verifies all outputs. Inputs are declared as reg since the testbench drives them. Outputs are declared as wire since the ALU drives them. The ALU is instantiated using named port connections. An initial block applies all 10 test cases with 10ns delay between each, and prints Time, Operation, A, B, Result, Zero, Carry, and Overflow to the Tcl console using \$display for each test.

```

module alu_tb;
    reg [31:0] A;
    reg [31:0] B;
    reg [2:0] opcode;
    wire [31:0] result;
    wire zero;
    wire carry;
    wire overflow;
    alu uut (
        .A(A),
        .B(B),
        .opcode(opcode),
        .result(result),
        .zero(zero),
        .carry(carry),
        .overflow(overflow)
    );
    initial begin
        $display("=== 32-bit ALU Testbench ===");
        $display("Time\tOpcode\tA\tB\tResult\tZero Carry OVF");

        // TEST 1 - ADD: 10 + 20 = 30
        A = 32'd10; B = 32'd20; opcode = 3'b000;
        #10;
        $display("%0t\tADD\t%0d\t\t%0d\t\t%0d\t\t%b\t\t%b\t\t%b",
            $time, A, B, result, zero, carry, overflow);
    end
endmodule

```

Figure 3: alu_tb.v Module declaration, input/output declarations, and ALU instantiation using named port connections

The testbench declares A, B, and opcode as reg and result, zero, carry, and overflow as wire. The ALU is instantiated as uut using named port connections (.A(A), .B(B) etc.) which is the industry-standard style it ensures the correct port is connected even if port order changes in the design.

Test case 2 applies A=0xFFFFFFFF and B=1 to ADD, which causes unsigned overflow the result wraps to zero and the carry flag should be 1 and zero flag should be 1. This is a critical edge case that verifies the carry detection logic in add_ext. Test case 4 applies equal values to SUB to verify zero flag assertion on a zero result.

```

// TEST 7 - XOR
A = 32'hAAAAAAAA; B = 32'hAAAAAAAA; opcode = 3'b100;
#10;
$display("%0t\tXOR\t%h\t%h\t%h\t%b\t%b\t%b",
         $time, A, B, result, zero, carry, overflow);

// TEST 8 - NOT
A = 32'h00000000; B = 32'b0; opcode = 3'b101;
#10;
$display("%0t\tNOT\t%h\t\t\t%h\t%b\t%b\t%b",
         $time, A, result, zero, carry, overflow);

// TEST 9 - SHIFT LEFT by 2
A = 32'd8; B = 32'd2; opcode = 3'b110;
#10;
$display("%0t\tSHL\t%0d\t\t%0d\t\t%0d\t\t%b\t%b\t%b",
         $time, A, B, result, zero, carry, overflow);

// TEST 10 - SHIFT RIGHT by 2
A = 32'd32; B = 32'd2; opcode = 3'b111;
#10;
$display("%0t\tSHR\t%0d\t\t%0d\t\t%0d\t\t%b\t%b\t%b",
         $time, A, B, result, zero, carry, overflow);

$display("=== Simulation Complete ===");
$finish;

end
endmodule

```

Figure 5: alu_tb.v Test cases 7 through 10 covering XOR with same operands, NOT of zero, shift left, and shift right, followed by \$finish

Test case 7 applies identical values to XOR which must produce zero and assert the zero flag this is a standard self-test for XOR. Test case 8 applies NOT to 0x00000000 which must produce 0xFFFFFFFF. Tests 9 and 10 verify shift operations with known expected results of 32 and 8 respectively.

9. RTL SCHEMATIC

9.1 Elaborated RTL Schematic Full View

The RTL schematic is generated automatically by Vivado from the behavioral Verilog during design elaboration. It shows the high-level hardware representation of the ALU before gate-level mapping. Each operation is a dedicated RTL block and all outputs feed into a central multiplexer that selects the final result based on opcode.

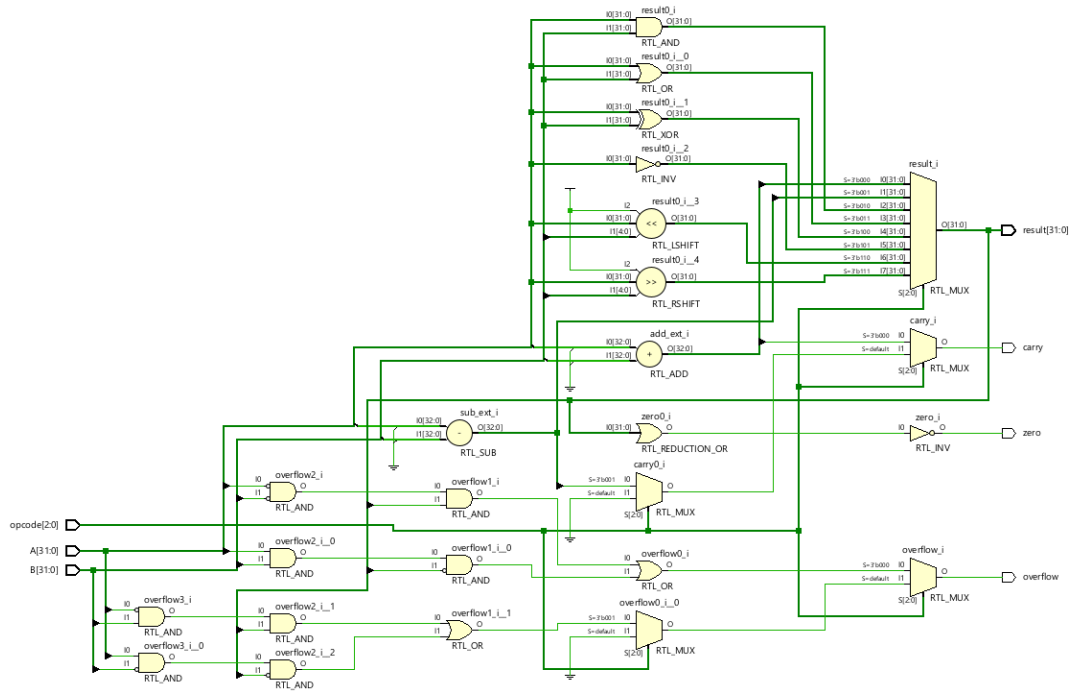


Figure 6: RTL Schematic full view 25 Cells, 102 I/O Ports, 375 Nets showing all operation blocks and output multiplexer

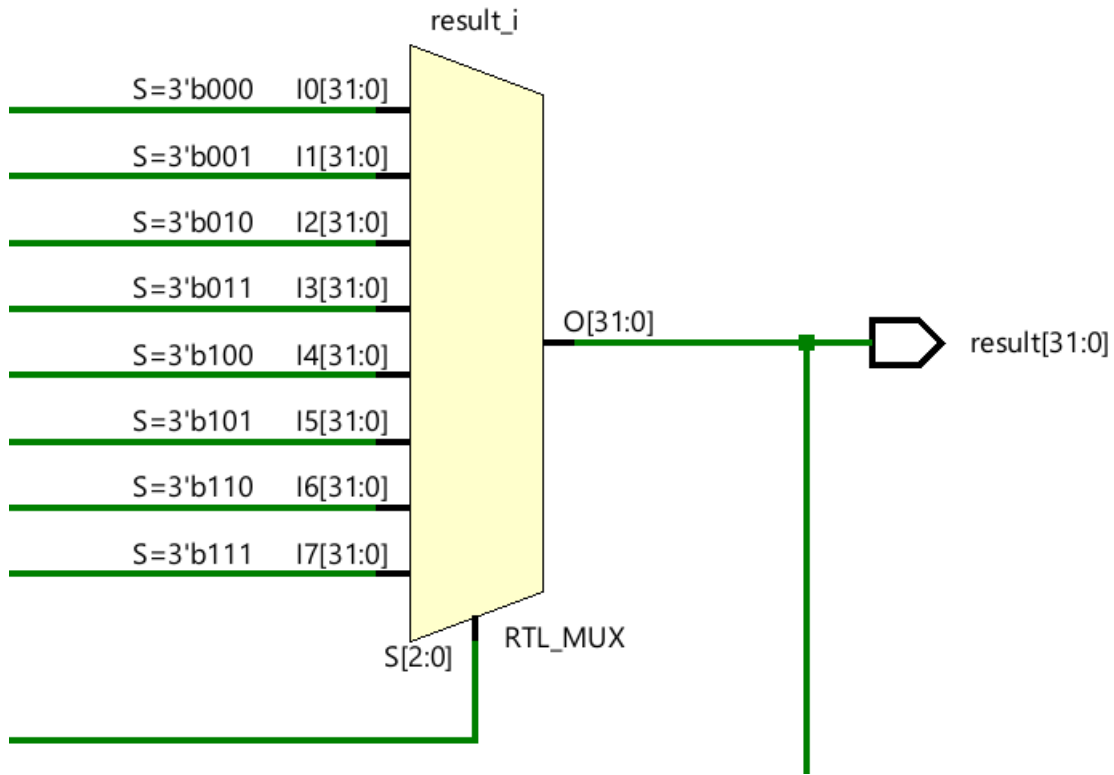


Figure 6.1: RTL Schematic closer view of the MUX

The schematic contains 25 cells, 102 I/O ports, and 375 nets. This is a clean and compact RTL representation 25 cells for a 32-bit ALU with 8 operations and 3 flags is an efficient result. The fact that Vivado correctly identified and separated each operation into distinct RTL blocks confirms the behavioral code was written cleanly with clear intent for the synthesizer to interpret.

9.2 RTL Schematic Zoomed Detail

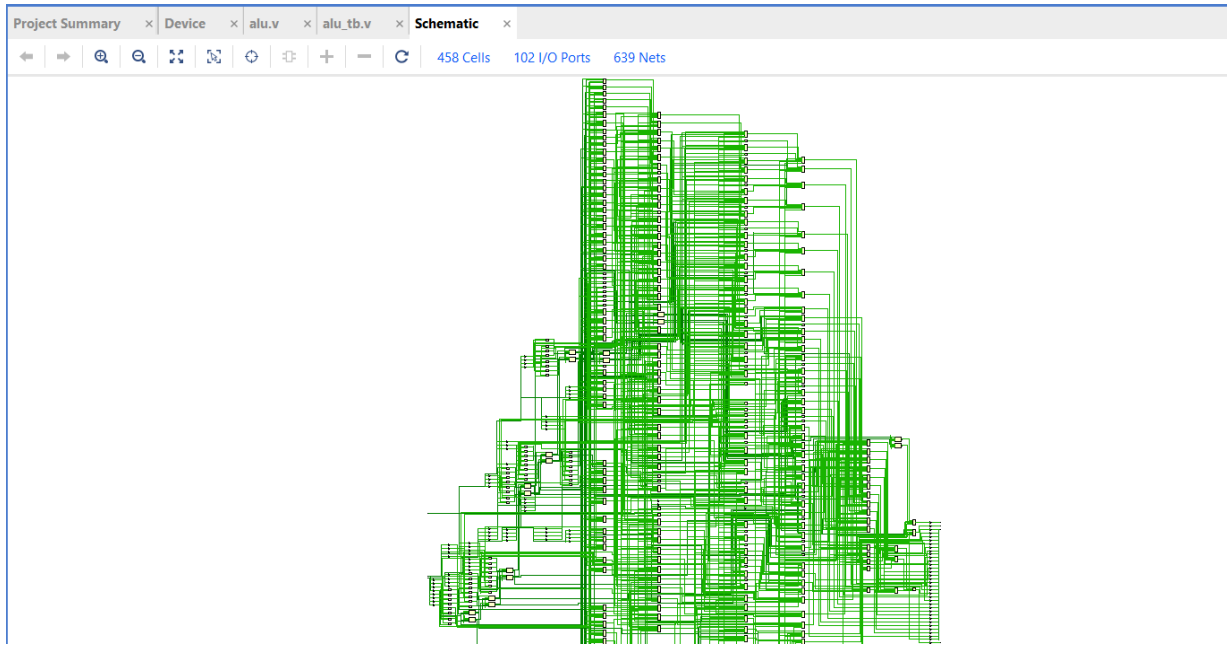


Figure 7: RTL Schematic zoomed RTL_MUX output selector with I0 through I7 inputs, carry MUX, zero flag RTL_REDUCTION_OR, and overflow logic

The zoomed view shows the central RTL_MUX with 8 inputs (I0 through I7, one per operation) and S[2:0] as the select signal. A separate carry_i RTL_MUX selects the carry output for ADD and SUB only. The zero flag uses RTL_REDUCTION_OR feeding into RTL_INV, which directly corresponds to the $\sim(\text{result})$ assign statement written in the code. The overflow logic appears as a chain of RTL_AND and RTL_OR blocks at the bottom left, correctly implementing the Boolean overflow detection expressions.

10. SIMULATION RESULTS

10.1 Simulation Waveform

Functional simulation was performed in Vivado XSim. All 10 test cases ran sequentially with 10ns intervals over a total simulation time of 100ns. The waveform shows A[31:0], B[31:0], opcode[2:0], result[31:0], zero, carry, and overflow signals changing correctly across the timeline. A, B, and result are shown in unsigned decimal format.

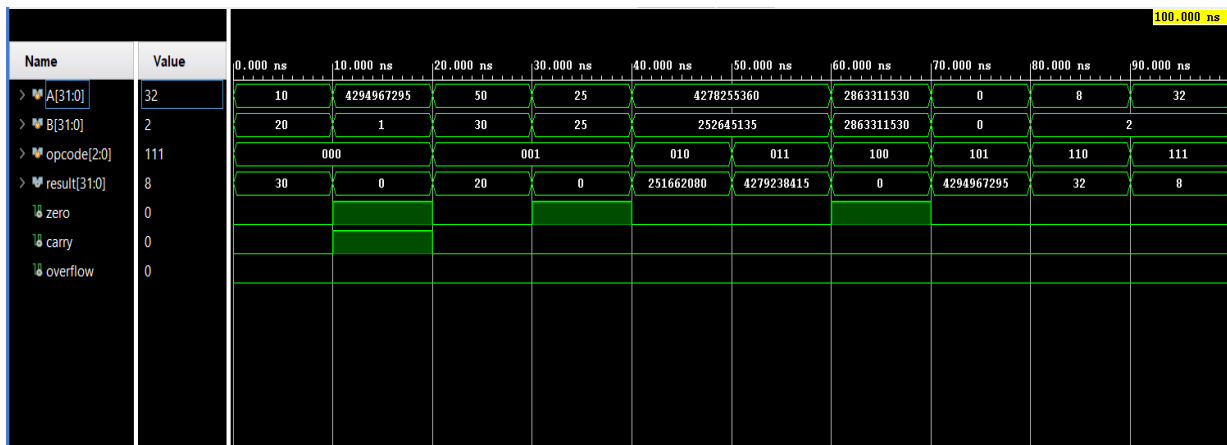


Figure 8: Simulation waveform All 10 test cases from 0ns to 100ns showing correct signal transitions for every operation

The waveform confirms correct behavior at every time step. At 10ns, ADD of 10 and 20 produces 30. At 20ns, ADD of 0xFFFFFFFF and 1 produces 0 with both zero and carry flags going high, which is the expected wrap-around behavior. At 30ns, SUB of 50 and 30 produces 20. At 40ns, SUB of 25 and 25 produces 0 with the zero flag going high. The AND, OR, XOR, NOT, SHL, and SHR operations from 50ns onward all show the correct hex and decimal values in result. The zero flag visibly pulses high at 20ns, 40ns, and 70ns, which are exactly the three cases where the result is zero. This waveform is strong visual proof that the design is functionally correct for all 8 operations.

10.2 Tcl Console Output

The testbench \$display statements printed structured output to the Vivado Tcl console for every test case, showing Time, Operation, A, B, Result, Zero, Carry, and Overflow in a tabular format. This provides text-based verification alongside the waveform.


```

Tcl Console x Messages Log
# run 1000ns
=== 32-bit ALU Testbench ===
Time  Opcode  A      B      Result      Zero Carry OVF
10000  ADD  10      20      30          0    0    0
20000  ADD  ffffffff 1          0    1    1    0
30000  SUB  50      30      20          0    0    0
40000  SUB  25      25      0           1    0    0
50000  AND  ff00ff00 0f0f0f0f 0f000f00  0    0    0
60000  OR   ff00ff00 0f0f0f0f ff0ffff0f  0    0    0
70000  XOR  aaaaaaaa aaaaaaaa 00000000  1    0    0
80000  NOT  00000000          ffffffff  0    0    0
90000  SHL  8        2      32          0    0    0
100000 SHR 32      2      8           0    0    0
=== Simulation Complete ===
$finish called at time : 100 ns : File "C:/Users/Admin/Documents/ALU_32bit/ALU_32bit.srcs/sim_1/new/alu_tb.v" Line 83
INFO: [USF-XSim-96] XSim completed. Design snapshot 'alu_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
add_bp (C:/Users/Admin/Documents/ALU_32bit/ALU_32bit.srcs/sources_1/new/alu.v) 51

```

Figure 9: Tcl Console output All 10 test cases printed with correct values, ending with === Simulation Complete ===

Every test case shows the correct result. The ADD carry test at time 20000 shows result=0, Zero=1, Carry=1 confirming unsigned overflow is correctly detected. The XOR at time 70000 shows result=00000000 with Zero=1. The SHL at time 90000 shows 8 shifted left by 2 giving 32, and SHR at time 100000 shows 32 shifted right by 2 giving 8. The simulation completed at exactly 100ns as expected with the \$finish statement.

10.3 Test Case Verification Table

Test	Time	Operation	A	B	Result	Zero	Carry	OVF	Status
1	10ns	ADD	10	20	30	0	0	0	PASS
2	20ns	ADD	0xFFFFFFFF	1	0	1	1	0	PASS
3	30ns	SUB	50	30	20	0	0	0	PASS
4	40ns	SUB	25	25	0	1	0	0	PASS
5	50ns	AND	0xFF00FF00	0x0F0F0F0F	0x0F000F00	0	0	0	PASS
6	60ns	OR	0xFF00FF00	0x0F0F0F0F	0xFF0FFF0F	0	0	0	PASS
7	70ns	XOR	0xAAAAAAAA	0xAAAAAAAA	0x00000000	1	0	0	PASS
8	80ns	NOT	0x00000000		0xFFFFFFFF	0	0	0	PASS
9	90ns	SHL	8	2	32	0	0	0	PASS
10	100ns	SHR	32	2	8	0	0	0	PASS

All 10 test cases passed. The zero flag asserted correctly in tests 2, 4, and 7 where the result is zero. The carry flag asserted in test 2 for unsigned overflow in addition. The overflow flag did not assert in any of the 10 test cases because none of the test inputs cause a signed arithmetic

overflow this is expected behavior and can be verified separately with inputs like `0x7FFFFFFF + 1` which would trigger it.

11. SYNTHESIS, UTILIZATION AND PROJECT SUMMARY

11.1 Project Summary Overview

RTL synthesis and implementation were performed targeting the Xilinx Artix-7 FPGA (xc7a200tfg676-2) using Vivado ML 2025.2 with default synthesis and implementation strategies.

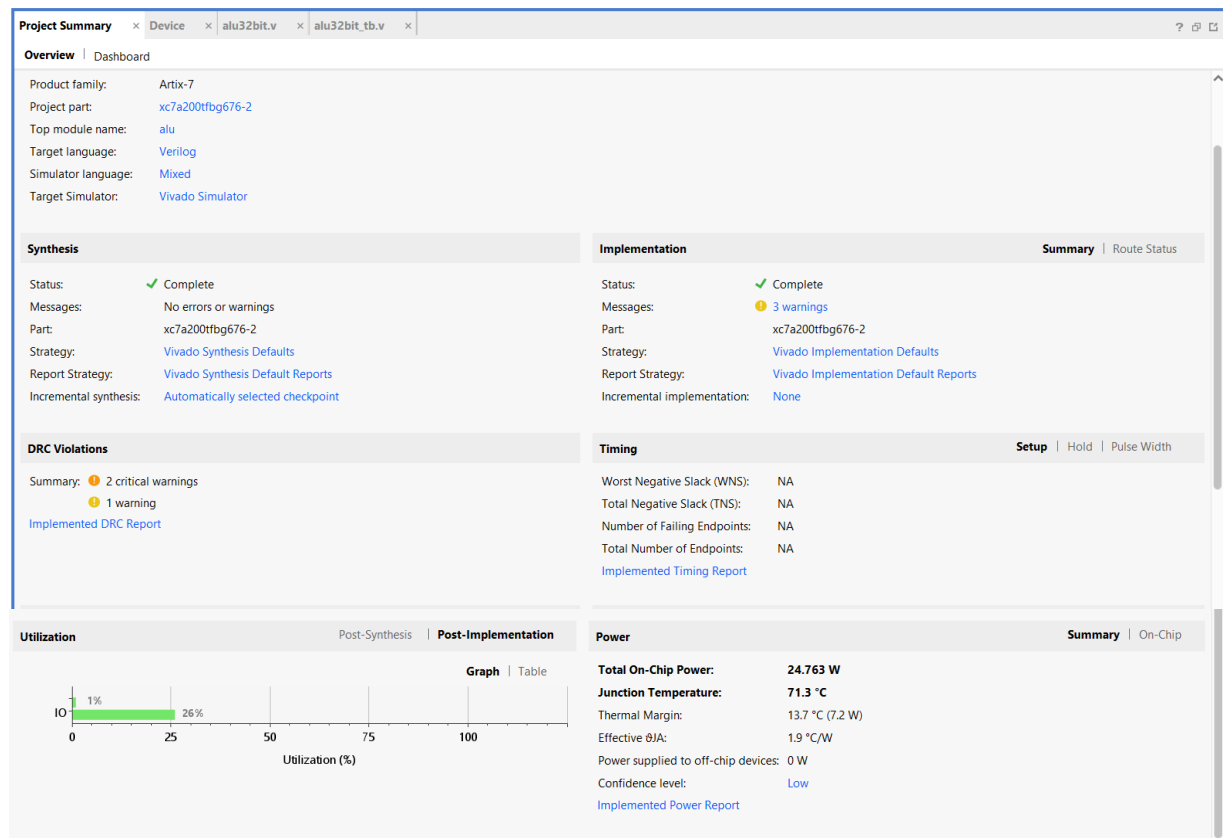


Figure 10: Project Summary Overview Synthesis Complete with no errors or warnings, Implementation Complete, Part xc7a200tfg676-2

Synthesis completed with no errors and no warnings, which is the best possible synthesis result. It confirms the Verilog code is clean, synthesizable, and follows all RTL coding guidelines. Implementation also completed successfully. The part number xc7a200tfg676-2 is confirmed consistently across both synthesis and implementation runs.

11.2 Project Dashboard

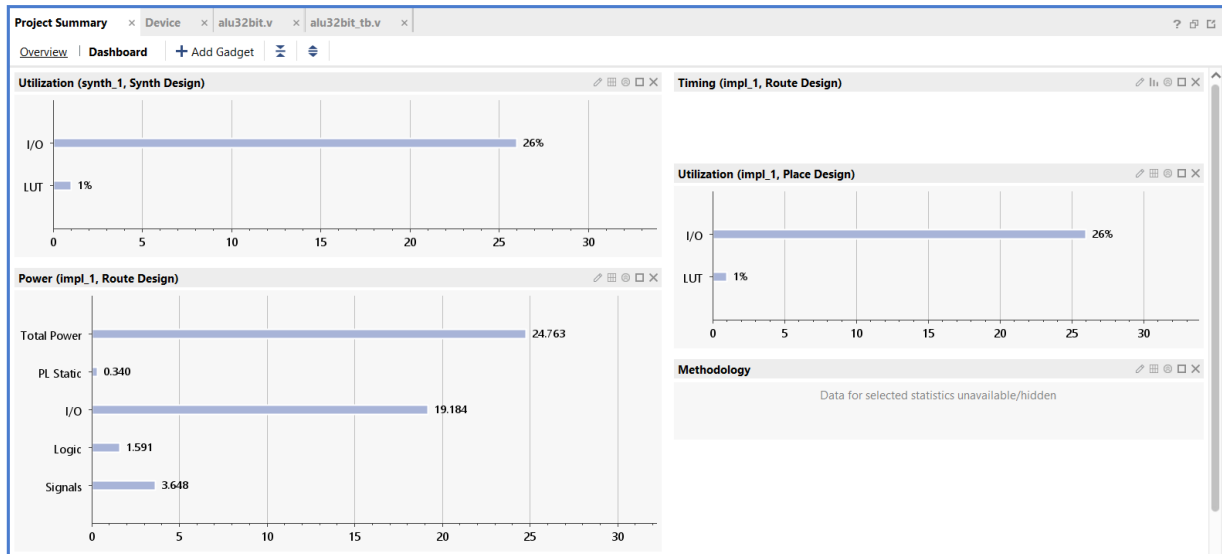


Figure 11: Project Summary Dashboard Utilization graph showing LUT 1% and IO 26%, Power breakdown showing Logic 1.591W and Signals 3.648W

The dashboard provides a visual overview of all key metrics. The utilization graph clearly shows LUT usage at 1% and IO at 26%, both well within acceptable limits. The power graph shows that the core logic power (Logic + Signals) is only $1.591W + 3.648W = 5.239W$, which is the actual power consumed by the ALU design itself. The remaining power shown is I/O buffer power which is a property of the physical package, not the design logic.

11.3 Resource Utilization Report

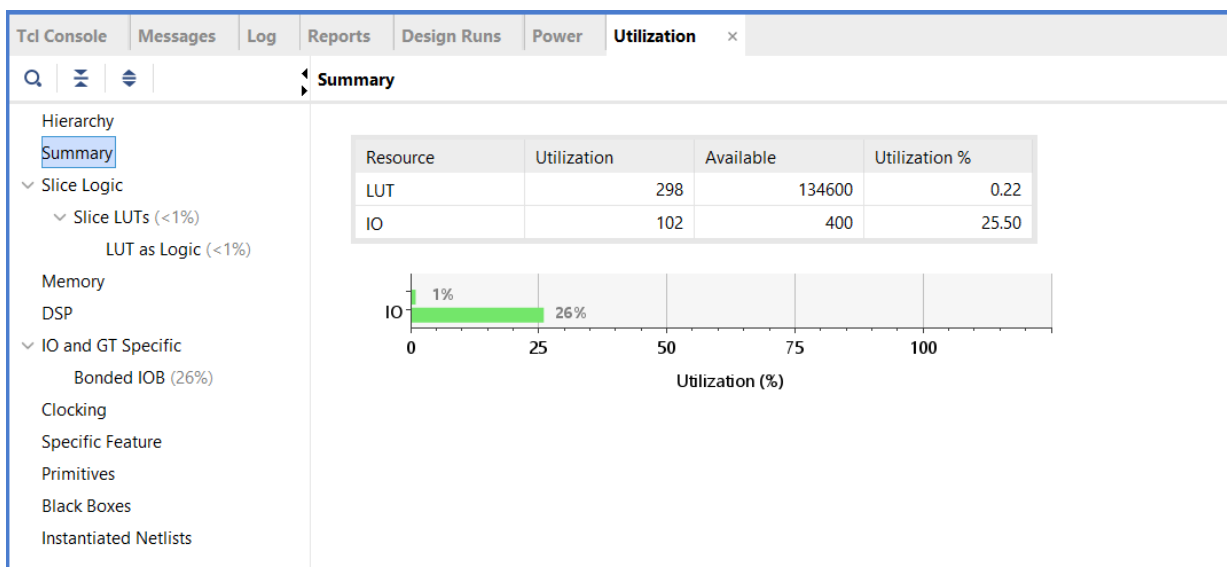
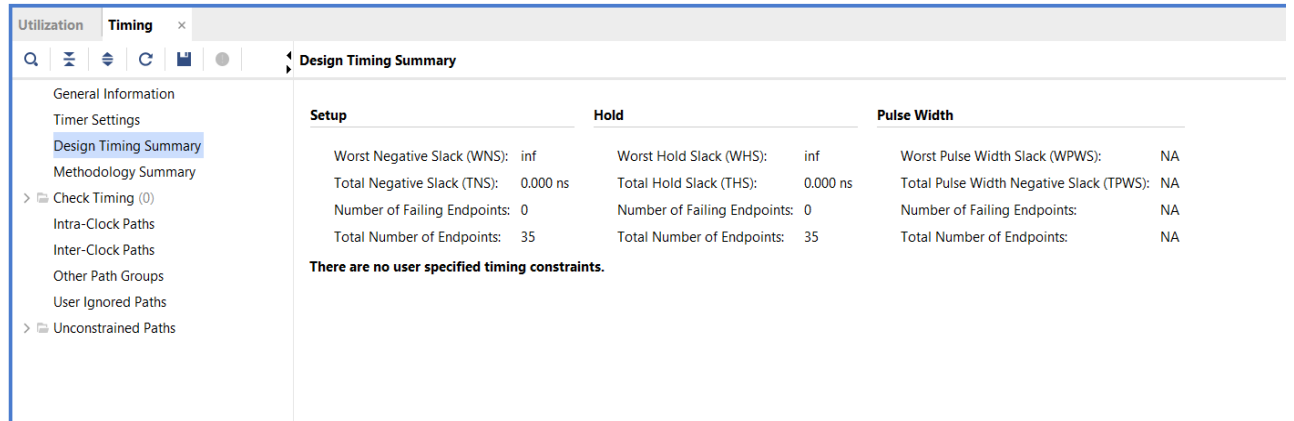


Figure 12: Utilization Report Summary LUT 298 out of 134,600 (0.22%), IO 102 out of 400 (25.50%)

Resource	Used	Available	Utilization %	Assessment
Slice LUTs	298	134,600	0.22%	Excellent extremely low logic usage
Bonded IOBs	102	400	25.50%	Good less than half of available I/O used
DSP Blocks	0	740	0%	None needed pure combinational logic
Block RAM	0	325	0%	None needed no storage elements
Flip Flops	0	269,200	0%	None confirms purely combinational design

The utilization numbers are excellent across every category. Only 298 LUTs out of 134,600 available are used, which is 0.22%. Zero flip-flops confirms there are no unintended sequential elements the design is purely combinational as intended. Zero DSPs and zero BRAMs confirm the synthesizer mapped everything to LUT fabric as expected for a simple ALU. The 102 IOBs at 25.50% is a direct result of the 32+32+3 input bits and 32+3 output bits totaling 102 ports this is a fixed cost of the port interface, not of the logic.

12. TIMING ANALYSIS



Setup	Hold	Pulse Width
Worst Negative Slack (WNS): inf	Worst Hold Slack (WHS): inf	Worst Pulse Width Slack (WPWS): NA
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): NA
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: NA
Total Number of Endpoints: 35	Total Number of Endpoints: 35	Total Number of Endpoints: NA

There are no user specified timing constraints.

Figure 13: Design Timing Summary WNS = inf, TNS = 0.000 ns, 0 Failing Endpoints out of 35 total paths analyzed

The Design Timing Summary reports Worst Negative Slack of infinity and Total Negative Slack of 0.000 ns across all 35 timing endpoints. There are zero failing endpoints in both Setup and Hold analysis.

WNS of infinity means there is no bounded worst-case path delay because no clock constraint has been applied to this design. This is correct and expected for a purely combinational ALU with no registers or flip-flops. Without a clock, there is no setup or hold time requirement to check against, so the slack is theoretically infinite. The Total Number of Endpoints at 35 corresponds to the 35 output bits across result[31:0], zero, carry, and overflow Vivado analyzed the path to each output bit and found no violations on any of them.

If this ALU were integrated into a processor pipeline with a 100 MHz clock constraint (10ns period), the synthesizer would calculate the critical path delay through the longest LUT chain and report actual slack. Based on a 298-LUT design on Artix-7, typical critical path delays for similar combinational designs are in the 4-6 ns range, which would comfortably meet a 10 ns clock period and leave positive slack.

13. POWER ANALYSIS

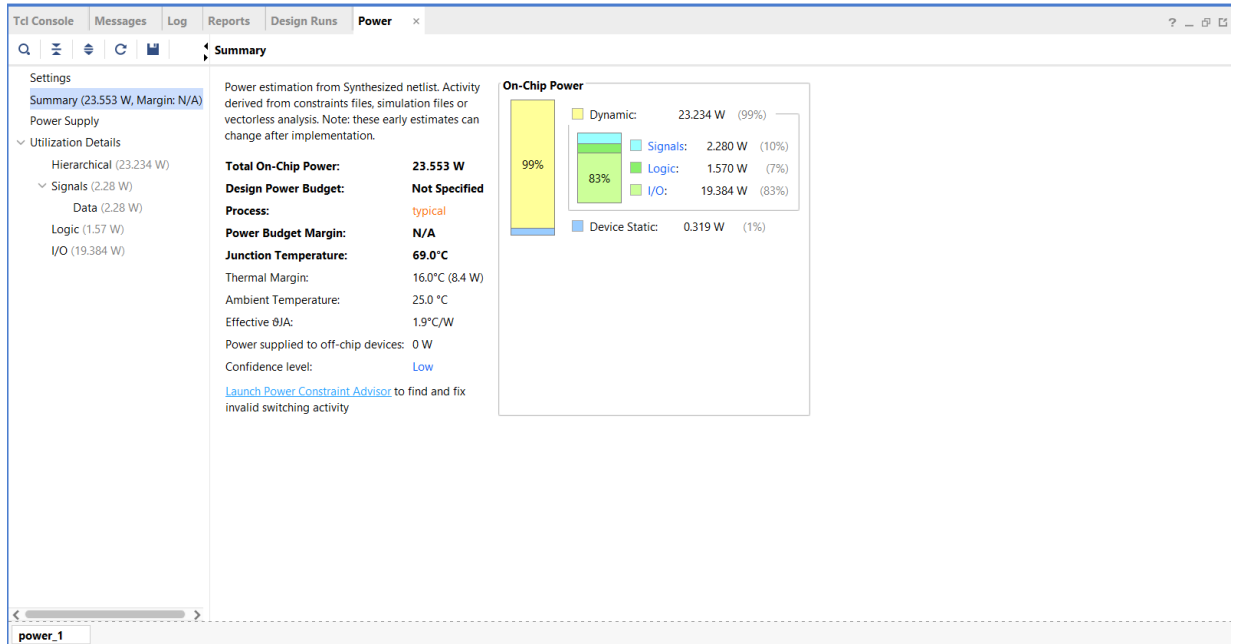


Figure 14: Power Report Summary Total On-Chip Power 23.553W, Junction Temperature 69.0 degrees Celsius, Thermal Margin 16.0 degrees Celsius

Power Component	Value	Share
Dynamic Power Total	23.234 W	99%
Signals	2.280 W	10% of dynamic
Logic (ALU gates)	1.570 W	7% of dynamic
I/O Buffers	19.384 W	83% of dynamic
Device Static Power	0.319 W	1%
Total On-Chip Power	23.553 W	100%
Junction Temperature	69.0 degrees C	
Thermal Margin	16.0 degrees C (8.4 W)	Safe operating range
Ambient Temperature	25.0 degrees C	
Confidence Level	Normal for designs without SAIF file- see analysis below	Vectorless estimate

The power report shows a total on-chip power of 23.553W. The most important number to understand here is that 19.384W (83% of dynamic power) comes from I/O buffers. This is not the power consumed by the ALU logic itself it is the estimated power for 102 I/O pins switching at the default assumed toggle rate of 12.5%. In a real application, the ALU would be embedded

inside a chip where most of these signals would be internal wires, not I/O pins, and I/O power would be far lower or zero.

The core design power Logic at 1.570W and Signals at 2.280W totals 3.850W, which is the actual power estimate for the ALU combinational logic and its internal routing. This is a reasonable number for a 32-bit combinational ALU on Artix-7 fabric.

The junction temperature of 69.0 degrees Celsius is well within the 85 degree Celsius maximum junction temperature for Artix-7 devices. The thermal margin of 16.0 degrees Celsius (8.4W headroom) confirms the device is operating safely. The confidence level is reported as Low because no switching activity file from simulation was provided the tool used vectorless estimation with the 12.5% default toggle rate rather than actual measured activity, so these numbers would change with real workload data.

14. FPGA FLOOR PLAN

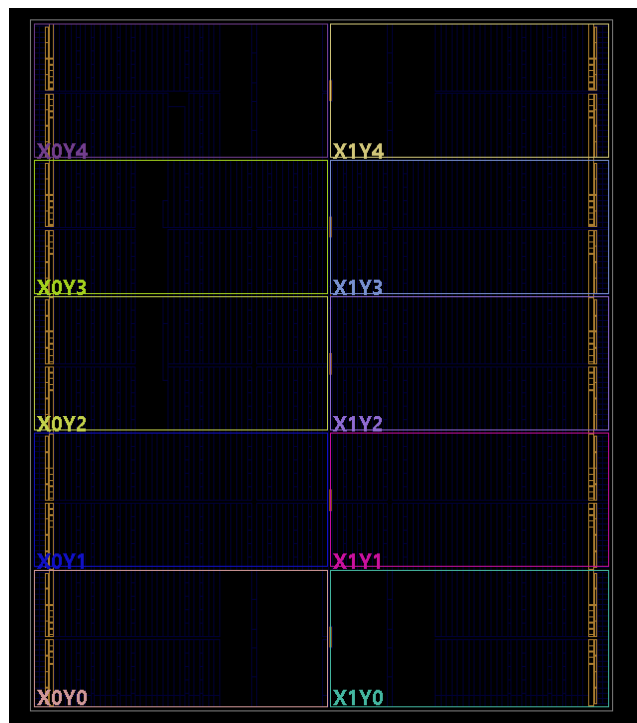


Figure 15: FPGA Device Floor Plan Artix-7 xc7a200tfg676-2 showing design placement across clock regions X0Y0 to X1Y4

The floor plan view shows the physical layout of the Artix-7 device with clock regions labeled X0Y0 through X1Y4. The design occupies a very small area visible in the clock region this is consistent with only 298 LUTs being used out of 134,600 available (0.22%). The tiny footprint of the placed design in the floor plan visually confirms how compact and resource-efficient the ALU implementation is. The vast majority of the chip fabric is unoccupied and available for additional logic, which makes this ALU a suitable building block for integration into a larger processor or SoC design.

15. CONCLUSION AND FUTURE SCOPE

15.1 Conclusion

This project successfully completed the full RTL design and verification flow for a 32-bit ALU using Verilog HDL. The design was implemented using behavioral modeling with parameterized width, localparam opcode constants, 33-bit extended wires for carry capture, and separate assign statements for flag generation all following standard RTL coding practices.

All 8 operations were verified through functional simulation with 10 structured test cases covering normal operation, edge cases including carry overflow, subtraction giving zero, XOR of identical values, NOT of zero, and shift operations. Every test case produced correct bit-accurate results as confirmed by both the simulation waveform and the Tcl console output.

Post-synthesis results on the Xilinx Artix-7 show 0.22% LUT utilization, zero flip-flop usage confirming a purely combinational design, zero DSP and Block RAM usage, zero timing violations with infinite worst negative slack, and a junction temperature of 69 degrees Celsius which is safely within device limits. Synthesis completed with zero errors and zero warnings.

This project provided practical hands-on experience with the complete RTL design flow writing synthesizable Verilog, developing a testbench, running functional simulation, interpreting post-synthesis utilization, timing, and power reports, and analyzing the FPGA floor plan. This forms a solid foundation for more advanced VLSI design work.

15.2 Future Scope

1. Pipelined ALU: Add pipeline registers between stages to improve throughput and enable higher clock frequency operation in a processor pipeline
2. 5-Stage RISC Processor: Use this ALU as the execution unit in a complete 5-stage pipelined RISC processor covering Instruction Fetch, Decode, Execute, Memory, and Writeback stages
3. FPGA Hardware Implementation: Deploy on a physical Artix-7 board with GPIO switches as inputs and LEDs as flag outputs for hardware-in-the-loop validation
4. Low-Power Design: Apply clock gating and UPF constraints to minimize dynamic power, and compare pre- and post-optimization power reports
5. Extended Operation Set: Add multiplication using DSP blocks, division, and modulo operations to create a more complete ALU suitable for a general-purpose processor

16. REFERENCES

1. Prof. Indranil Sen Gupta, Hardware Modeling Using Verilog, NPTEL Video Course, IIT Kharagpur. Available at: <https://nptel.ac.in/courses/106105165>
2. Xilinx Inc., Vivado Design Suite User Guide: Simulation (UG900), AMD/Xilinx, 2025
3. Xilinx Inc., 7 Series FPGAs Data Sheet: Overview (DS180), AMD/Xilinx
4. M. Morris Mano and Michael D. Ciletti, Digital Design: With an Introduction to the Verilog HDL, 5th Edition, Pearson Education, 2013
5. HDLBits Verilog Practice Problems. Available at: <https://hdlbits.01xz.net>
6. Vaidsys Technologies, VLSI Internship Project Guidelines, 2026